

Introduzione al CSS

```
/**  
 * @author Rocco Mazzeo  
 * @email rocco.mazzeo@gmail.com  
 * @linkedin https://www.linkedin.com/in/roccomazzeo  
 */
```

Cos'è il CSS ?

Il CSS (Cascading Style Sheets) permette di separare la struttura (HTML) dalla presentazione (stili) di una pagina web. Consente di controllare colori, spaziature, layout, tipografia e molto altro.

Vantaggi dell'uso del CSS

- **Manutenzione facilitata:** Modificare lo stile di più pagine cambiando un solo file CSS.
- **Consistenza:** Garantisce un aspetto uniforme su tutto il sito.
- **Accessibilità:** Migliora l'esperienza utente su diversi dispositivi e per utenti con disabilità.
- **Prestazioni:** Riduce la quantità di codice duplicato e velocizza il caricamento delle pagine.

Esempio di base

```
body {  
  background-color: #f0f0f0;  
  color: #333;  
}
```

Esercitazione: Applicare uno sfondo rosso e testo nero

Crea una semplice pagina HTML e applica uno stile CSS per avere lo sfondo rosso e il testo nero.

Esempio:

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Esercitazione CSS</title>
    <style>
      body {
        background-color: red;
        color: black;
      }
    </style>
  </head>
  <body>
    <h1>Benvenuto!</h1>
    <p>Questa pagina ha uno sfondo rosso e il testo nero.</p>
  </body>
</html>
```

Sintassi di base

Selettori, proprietà e valori: la struttura di una regola CSS.

Struttura di una regola CSS

Una regola CSS è composta da:

- **Selettore:** indica a quali elementi HTML applicare lo stile.
- **Dichiarazione:** racchiusa tra parentesi graffe `{ }` e composta da una o più coppie proprietà/valore.
- **Proprietà:** l'aspetto da modificare (es. `color` , `font-size`).
- **Valore:** il valore assegnato alla proprietà (es. `red` , `16px`).

Esempio:

```
p {  
  color: blue;  
  font-size: 18px;  
}
```

In questo esempio, tutti gli elementi `<p>` avranno il testo blu e una dimensione di 18px.

ID e classi nell'HTML

- **ID:** Attributo che identifica un elemento in modo univoco nella pagina. Si usa per applicare stili specifici o per riferirsi all'elemento tramite JavaScript.

```
<h1 id="intestazione">Titolo principale</h1>
```

- **Classe:** Attributo che permette di raggruppare più elementi con lo stesso stile. Un elemento può avere più classi separate da spazi.

```
<p class="evidenziato importante">Testo evidenziato</p>
```

Gli ID si usano per selezionare un singolo elemento, mentre le classi servono per applicare lo stesso stile a più elementi.

Proprietà CSS più utilizzate

Ecco alcune delle proprietà CSS più comuni con una breve descrizione:

- **color:** imposta il colore del testo.

Esempio: `color: red;` , `color: #333;` , `color: rgb(0,0,0);`

- **background-color:** definisce il colore di sfondo di un elemento.

Esempio: `background-color: yellow;` , `background-color: #f0f0f0;`

- **font-size:** specifica la dimensione del testo.

Esempio: `font-size: 16px;` , `font-size: 1.2em;`

- **font-family:** indica il tipo di carattere da utilizzare.

Esempio: `font-family: Arial, sans-serif;`

- **font-weight:** determina lo spessore del testo (es. bold).

Esempio: `font-weight: bold;` , `font-weight: 700;`

- **text-align:** allinea il testo (sinistra, destra, centro, giustificato).

Esempio: `text-align: center;` , `text-align: right;`

- **margin:** imposta lo spazio esterno attorno a un elemento.

Esempio: `margin: 20px;` , `margin: 10px 5px;`

- **padding:** definisce lo spazio interno tra il bordo e il contenuto di un elemento.

Esempio: `padding: 15px;` , `padding: 5px 10px;`

- **border:** aggiunge un bordo attorno a un elemento (spessore, stile, colore).

Esempio: `border: 1px solid black;` , `border: 2px dashed #ccc;`

- **width / height:** specificano larghezza e altezza di un elemento.

Esempio: `width: 100px;` , `height: 50%;`

- **display:** controlla il tipo di box di un elemento (block, inline, flex, ecc.).
Esempio: `display: block; , display: flex;`
- **position:** definisce il metodo di posizionamento (static, relative, absolute, fixed, sticky).
Esempio: `position: absolute; , position: fixed;`
- **top / right / bottom / left:** posizionano un elemento rispetto al suo contenitore.
Esempio: `top: 10px; , left: 0;`
- **overflow:** gestisce il contenuto che esce dai limiti dell'elemento (visible, hidden, scroll, auto).
Esempio: `overflow: hidden; , overflow: auto;`

- **z-index:** imposta la profondità di sovrapposizione degli elementi posizionati.

Esempio: `z-index: 10;`

- **opacity:** regola la trasparenza dell'elemento.

Esempio: `opacity: 0.5;`

- **box-shadow:** aggiunge un'ombra al box dell'elemento.

Esempio: `box-shadow: 2px 2px 8px #888;`

- **text-shadow:** aggiunge un'ombra al testo.

Esempio: `text-shadow: 1px 1px 2px gray;`

Queste proprietà sono la base per la maggior parte delle personalizzazioni CSS.

Risorsa: Elenco completo delle proprietà CSS

Per consultare la lista aggiornata e ufficiale di tutte le proprietà CSS, visita la documentazione MDN:

- [Elenco completo delle proprietà CSS su MDN Web Docs](#)

Questa pagina contiene descrizioni, esempi e dettagli di compatibilità per ogni proprietà.

Selettori CSS

Selettori di tipo, classe, ID e universali.

- **Selettore di tipo:** seleziona tutti gli elementi di un certo tipo.

```
h1 {  
  color: green;  
}
```

Tutti gli `<h1>` saranno verdi.

- **Selettore di classe:** seleziona tutti gli elementi con una certa classe.

```
.evidenziato {  
  background-color: yellow;  
}
```

Tutti gli elementi con `class="evidenziato"` avranno sfondo giallo.

- **Selettore di ID:** seleziona l'elemento con un certo ID (unico nella pagina).

```
#intestazione {  
  font-size: 24px;  
}
```

L'elemento con `id="intestazione"` avrà dimensione testo 24px.

- **Selettore universale:** seleziona tutti gli elementi della pagina.

```
* {  
  margin: 0;  
  padding: 0;  
}
```

Tutti gli elementi avranno margini e padding azzerati.

Esercitazione: Sperimenta con i selettori CSS

Crea una pagina HTML con alcuni elementi che abbiano ID, classi e tipi diversi. Scrivi un foglio di stile CSS che:

1. Renda tutti i titoli `<h2>` di colore blu.
2. Applichi uno sfondo giallo agli elementi con la classe `evidenziato`.
3. Imposti la dimensione del testo a 28px per l'elemento con ID `intestazione`.
4. Azzeri margini e padding per tutti gli elementi della pagina.

Esempio di struttura HTML:

```
<h2 id="intestazione">Titolo della pagina</h2>  
<p class="evidenziato">Questo paragrafo è evidenziato.</p>  
<p>Questo paragrafo non ha classi.</p>
```

Selettori avanzati

Selettori discendenti, figli, fratelli adiacenti e generali.

Esempi di selettori avanzati

- **Selettore discendente:** seleziona tutti gli elementi che sono discendenti di un altro elemento.

```
nav ul li {  
  list-style-type: square;  
}
```

Tutti gli `<li>` dentro un `<ul>` dentro un `<nav>` avranno il pallino quadrato.

- **Selettore figlio diretto:** seleziona solo gli elementi figli diretti.

```
div > p {  
  color: purple;  
}
```

Solo i `<p>` figli diretti di un `<div>` saranno viola.

- **Selettore fratello adiacente:** seleziona l'elemento che segue immediatamente un altro.

```
h2 + p {  
  margin-top: 0;  
}
```

Il `<p>` subito dopo un `<h2>` non avrà margine superiore.

- **Selettore fratello generale:** seleziona tutti gli elementi fratelli successivi.

```
h2 ~ p {  
  color: teal;  
}
```

Tutti i `<p>` dopo un `<h2>` avranno il testo color teal.

Esercitazione: Selettori avanzati

Crea una pagina HTML che contenga una struttura simile a questa:

```
<nav>
  <ul>
    <li>Home</li>
    <li>Chi siamo</li>
    <li>Contatti</li>
  </ul>
</nav>
<div>
  <p>Paragrafo 1</p>
  <p>Paragrafo 2</p>
  <span>Testo span</span>
  <p>Paragrafo 3</p>
</div>
<h2>Titolo</h2>
<p>Paragrafo dopo h2</p>
<p>Altro paragrafo dopo h2</p>
```

Scrivi le seguenti regole CSS utilizzando i selettori avanzati:

1. Imposta il tipo di elenco quadrato solo per gli `<li>` dentro un `<nav>` .
2. Rendi viola solo i `<p>` figli diretti di un `<div>` .
3. Rimuovi il margine superiore solo dal `<p>` che segue immediatamente un `<h2>` .
4. Cambia il colore del testo in teal per tutti i `<p>` che seguono un `<h2>` (anche non immediatamente).

Suggerimento: Usa i selettori `nav ul li`, `div > p`, `h2 + p`, `h2 ~ p` .

Selettori di attributo

Come selezionare elementi in base agli attributi HTML.

Esempi di selettori di attributo

- **Selettore per attributo presente:** seleziona tutti gli elementi che hanno un certo attributo.

```
input[required] {  
  border: 2px solid red;  
}
```

Tutti gli `<input>` con l'attributo `required` avranno un bordo rosso.

- **Selettore per valore esatto:** seleziona elementi con un attributo che ha un valore specifico.

```
a[target="_blank"] {  
  color: orange;  
}
```

Tutti i link che si aprono in una nuova scheda saranno arancioni.

- **Selettore per valore che inizia con:** seleziona elementi il cui attributo inizia con un certo valore.

```
img[src^="https://"] {  
  border: 1px solid green;  
}
```

Tutte le immagini caricate da URL HTTPS avranno un bordo verde.

- **Selettore per valore che termina con:** seleziona elementi il cui attributo termina con un certo valore.

```
a[href$=".pdf"] {  
  text-decoration: underline;  
}
```

Tutti i link ai file PDF saranno sottolineati.

- **Selettore per valore che contiene:** seleziona elementi il cui attributo contiene una certa stringa.

```
div[class*="box"] {  
  background-color: #eee;  
}
```

Tutti i `<div>` con una classe che contiene la parola "box" avranno uno sfondo grigio chiaro.

Pseudo-classi

:hover, :active, :focus e altre pseudo-classi utili.

Esempi di pseudo-classi

- **:hover** – Applica uno stile quando il mouse passa sopra un elemento.

```
a:hover {  
  text-decoration: underline;  
  color: red;  
}
```

- **:active** – Applica uno stile quando l'elemento è attivo (ad esempio, durante il click).

```
button:active {  
  background-color: #ccc;  
}
```

- **:focus** – Applica uno stile quando l'elemento ha il focus (ad esempio, un campo input selezionato).

```
input:focus {  
  border-color: blue;  
  outline: none;  
}
```

- **:nth-child(n)** – Seleziona il(n)-esimo figlio di un elemento.

```
li:nth-child(odd) {  
  background-color: #f9f9f9;  
}
```

Queste pseudo-classi permettono di aggiungere interattività e stili dinamici agli elementi HTML.

Pseudo-elementi

::before, ::after, ::first-line, ::first-letter.

Colori in CSS

Formato	Esempio CSS	Descrizione
Nome	<code>color: red;</code>	Nome predefinito (es. red, blue, green)
HEX	<code>color: #ff0000;</code>	Codice esadecimale (3 o 6 cifre)
RGB	<code>color: rgb(255,0,0);</code>	Rosso, verde, blu (0–255)
RGBA	<code>color: rgba(255,0,0,0.5);</code>	RGB + trasparenza (0–1)
HSL	<code>color: hsl(0,100%,50%);</code>	Tonalità, saturazione, luminosità
HSLA	<code>color: hsla(0,100%,50%,0.5);</code>	HSL + trasparenza (0–1)

HSL: tonalità (0–360), saturazione (0–100%), luminosità (0–100%).

Unità di misura in CSS (parte 1)

Unità	Esempio CSS	Descrizione
px	<code>font-size: 16px;</code>	Pixel: unità fissa, non scalabile, indipendente dalla dimensione del dispositivo.
em	<code>font-size: 2em;</code>	Relativa alla dimensione del font dell'elemento padre. Se il font del genitore è 16px, <code>2em</code> equivale a 32px.

Nota: Le unità relative come `em` permettono una maggiore flessibilità e adattabilità del layout.

Unità di misura in CSS (parte 2)

Unità	Esempio CSS	Descrizione
rem	<code>font-size: 1.5rem;</code>	Relativa alla dimensione del font dell'elemento radice (<code>html</code>). Se <code>html</code> ha <code>font-size: 16px</code> , allora <code>1.5rem</code> = 24px. Consigliata per coerenza su tutta la pagina.
%	<code>width: 50%;</code>	Percentuale rispetto all'elemento contenitore. Utile per layout fluidi e responsivi.

Le unità relative come `rem` e `%` sono utili per layout scalabili e responsivi.

Unità di misura in CSS (parte 3)

Unità	Esempio CSS	Descrizione
vw	<code>width: 80vw;</code>	Percentuale della larghezza della finestra (viewport width). <code>100vw</code> corrisponde all'intera larghezza visibile del browser.
vh	<code>height: 50vh;</code>	Percentuale dell'altezza della finestra (viewport height). <code>100vh</code> è l'altezza totale della finestra.
fr	<code>grid-template-columns: 1fr 2fr;</code>	Unità frazionaria usata in CSS Grid per dividere lo spazio disponibile tra le colonne o righe.
ch	<code>width: 40ch;</code>	Larghezza relativa alla larghezza del carattere "0" del font corrente. Utile per controllare la larghezza dei blocchi di testo.

Consiglio: Le unità viewport (`vw` , `vh`), `fr` e `ch` sono fondamentali per design responsivi e layout avanzati.

Box Model

Margini, bordi, padding e contenuto.

Approfondimento: Box Model CSS

Il **Box Model** è il concetto fondamentale che descrive come gli elementi HTML vengono visualizzati e dimensionati nel browser. Ogni elemento è rappresentato come un rettangolo composto da quattro parti principali:

1. Content (Contenuto):

L'area dove viene visualizzato il testo, le immagini o altri contenuti.

2. Padding (Spaziatura interna):

Lo spazio tra il contenuto e il bordo. Si può impostare con la proprietà `padding`.

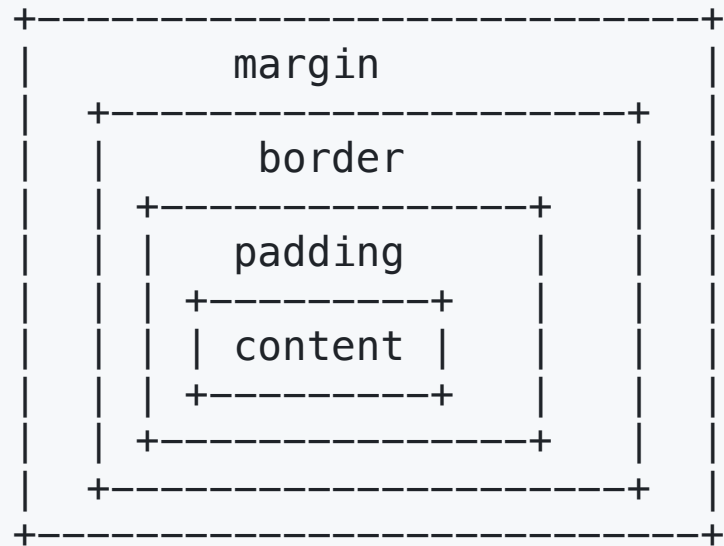
3. Border (Bordo):

La linea che circonda il padding e il contenuto. Si controlla con le proprietà `border-width`, `border-style`, `border-color`.

4. Margin (Spaziatura esterna):

Lo spazio tra il bordo dell'elemento e gli altri elementi. Si imposta con la proprietà `margin`.

Schema Box Model



Proprietà principali

- `width` e `height` : dimensioni del contenuto.
- `padding` : spazio interno (es. `padding: 10px 20px;`).
- `border` : bordo attorno al padding (es. `border: 2px solid #333;`).
- `margin` : spazio esterno (es. `margin: 15px;`).

Esempio pratico

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 3px solid #007bff;  
  margin: 30px;  
  background-color: #f9f9f9;  
}
```

Box-sizing

Di default, `width` e `height` si riferiscono solo all'area del contenuto.

Con `box-sizing: border-box;`, padding e bordo sono inclusi nelle dimensioni totali dell'elemento:

```
.box {  
  box-sizing: border-box;  
}
```

Questa proprietà semplifica la gestione dei layout, evitando calcoli manuali sulle dimensioni.

Riepilogo

- Il Box Model influenza il layout e la spaziatura degli elementi.
- Comprendere padding, border e margin è essenziale per creare layout precisi e responsivi.
- Usare `box-sizing: border-box` è una best practice moderna per evitare problemi di dimensionamento.

Proprietà di layout

display, position, float, clear.

Approfondimento: Proprietà `display`

La proprietà `display` determina come un elemento viene generato nel flusso del documento.

Valore	Descrizione
block	Occupi tutta la larghezza disponibile.
inline	Occupi solo lo spazio necessario.
inline-block	Come inline, ma accetta larghezza/altezza.
flex	Attiva il layout Flexbox.
grid	Attiva il layout Grid.
none	Nasconde l'elemento.

Esempio:

```
span {  
  display: inline-block;  
  width: 100px;  
}
```

Approfondimento: Proprietà `position`

La proprietà `position` controlla il posizionamento degli elementi.

Valore	Descrizione
static	Posizionamento normale nel flusso (default).
relative	Posizionato rispetto alla posizione originale.
absolute	Posizionato rispetto al primo antenato posizionato.
fixed	Fisso rispetto alla finestra del browser.
sticky	Si comporta come relative, poi come fixed quando si raggiunge una soglia.

Esempio:

```
.box {  
  position: absolute;  
  top: 20px;  
  left: 40px;  
}
```

Approfondimento: Proprietà **float** e **clear**

- **float**: Permette di spostare un elemento a sinistra o destra, facendo "galleggiare" il contenuto accanto.
- **clear**: Impedisce che elementi galleggianti si dispongano accanto.

Esempio:

```
img {  
  float: left;  
  margin-right: 20px;  
}  
p {  
  clear: both;  
}
```

Esercitazione: Layout con display, position e float

Crea una pagina HTML con:

- Un header in alto (display: block).
- Un menu laterale (float: left).
- Un contenuto principale (margin-left per lasciare spazio al menu).
- Un footer fissato in basso (position: fixed; bottom: 0; width: 100%).

Scrivi le regole CSS per ottenere questo layout.

Esempio HTML + CSS:

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Layout con display, position e float</title>
    <link rel="stylesheet" href="stile-layout.css" />
  </head>
  <body>
    <header>Header</header>
    <nav>
      <ul>
        <li>Home</li>
        <li>Chi siamo</li>
        <li>Contatti</li>
      </ul>
    </nav>
    <main>
      <h1>Contenuto principale</h1>
      <p>Questa è la sezione principale della pagina.</p>
    </main>
    <footer>Footer fissato in basso</footer>
  </body>
</html>
```

File CSS (`style-layout.css`):

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
}
header {
  background: #007bff;
  color: #fff;
  padding: 20px;
  text-align: center;
  font-size: 1.5em;
  display: block;
}
nav {
  background: #f0f0f0;
  width: 200px;
  float: left;
  height: calc(100vh - 100px);
  padding: 20px 0;
}
main {
  margin-left: 220px;
  padding: 20px;
  min-height: 300px;
}
footer {
  background: #333;
  color: #fff;
  position: fixed;
  bottom: 0;
  left: 0;
  width: 100%;
  padding: 15px;
  text-align: center;
}
```

Esercitazione extra: Layout responsive con menu mobile

Aggiungi un menu dedicato al mobile che si vede solo su dispositivi piccoli e nasconde quello desktop.

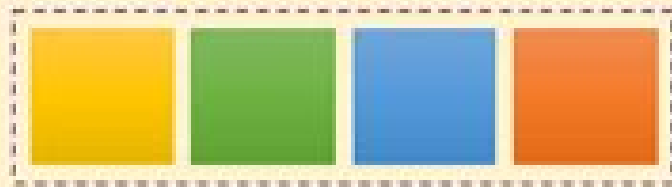
JavaScript (per aprire/chiusure il menu mobile):

```
document.getElementById("open-menu").onclick = function () {  
    var list = document.getElementById("mobile-list");  
    list.style.display = list.style.display === "block" ? "none" : "block";  
};
```

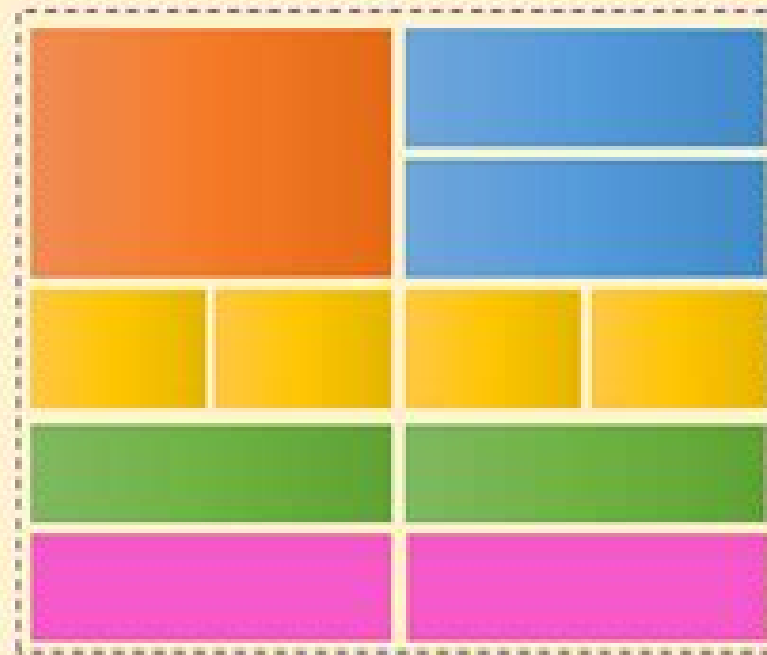
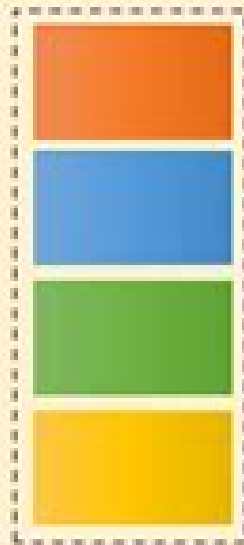
Nota:

- Il menu desktop (`.menu-desktop`) è visibile solo su schermi larghi.
- Il menu mobile (`.menu-mobile`) appare solo su schermi piccoli.
- Il bottone mostra/nasconde la lista delle voci di menu su mobile.

Flexbox - Grid



**CSS
FLEXBOX**



CSS GRID

Flexbox

Concetti base: container, item, proprietà principali.

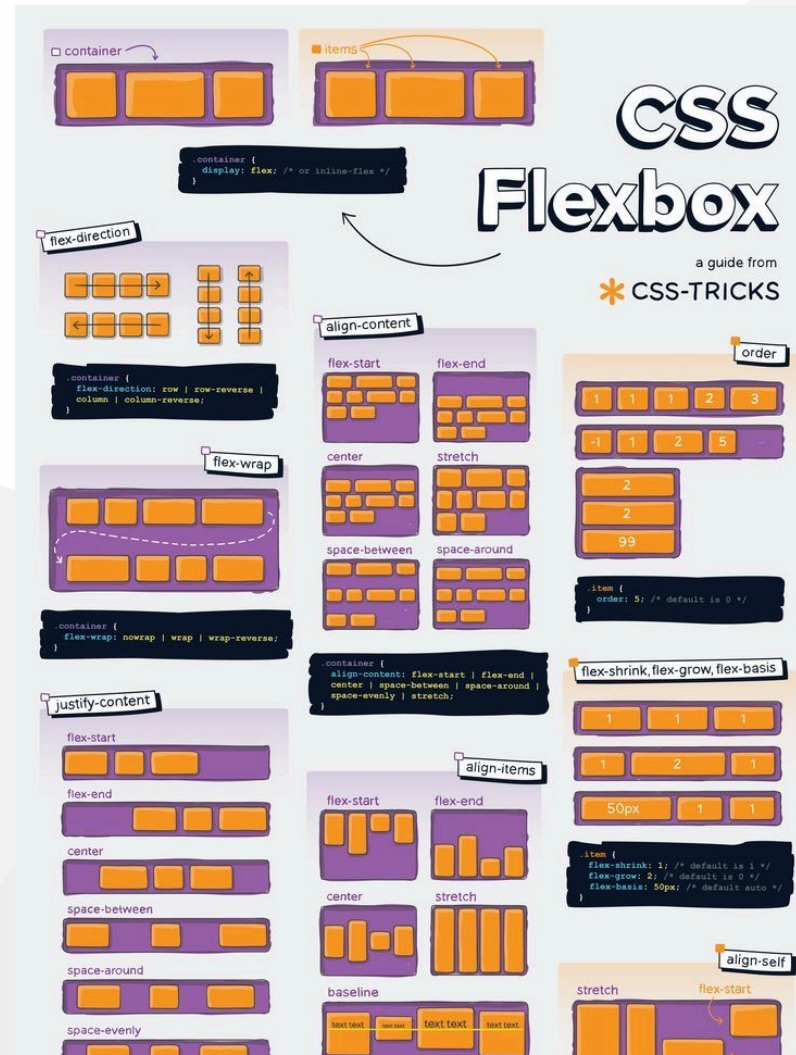
Cos'è Flexbox?

Flexbox (Flexible Box Layout) è un sistema di layout CSS che permette di distribuire lo spazio tra gli elementi di un contenitore e di allinearli facilmente, anche quando la loro dimensione è sconosciuta o dinamica.

[CSS Tricks - Flexbox](#)

Anteprima visiva: come funziona Flexbox

[Immagine su Pinterest](#)



Struttura di base

Per usare Flexbox, si imposta `display: flex` sul contenitore. Gli elementi figli diventano "flex item".

Esempio:

```
.container {  
  display: flex;  
}  
.item {  
  background: #eee;  
  padding: 20px;  
  margin: 10px;  
}
```

```
<div class="container">  
  <div class="item">Elemento 1</div>  
  <div class="item">Elemento 2</div>  
  <div class="item">Elemento 3</div>  
</div>
```

Proprietà principali del container

- `display: flex` – attiva Flexbox sul contenitore.
- `flex-direction` – imposta la direzione degli item (`row` , `column` , ecc.).
- `justify-content` – allinea gli item orizzontalmente.
- `align-items` – allinea gli item verticalmente.
- `flex-wrap` – permette agli item di andare a capo se non c'è spazio.

Esempio pratico

```
.flex-demo {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
  gap: 20px;  
}
```

```
<div class="flex-demo">  
  <div>Box 1</div>  
  <div>Box 2</div>  
  <div>Box 3</div>  
</div>
```

Esercitazione: Layout con Flexbox

Crea una barra di navigazione orizzontale con Flexbox:

```
<nav class="navbar">
  <a href="#">Home</a>
  <a href="#">Servizi</a>
  <a href="#">Contatti</a>
</nav>
```

```
.navbar {
  display: flex;
  justify-content: space-around;
  background: #007bff;
  padding: 15px;
}
.navbar a {
  color: #fff;
  text-decoration: none;
  font-weight: bold;
}
```

Flexbox: proprietà container

flex-direction, flex-wrap, justify-content, align-items, align-content.

Dettaglio delle proprietà del container Flexbox

- **flex-direction:**

Imposta la direzione principale degli item nel container.

- `row` (default): da sinistra a destra
- `row-reverse` : da destra a sinistra
- `column` : dall'alto in basso
- `column-reverse` : dal basso in alto

Esempio:

```
.container {  
  flex-direction: column;  
}
```

- **flex-wrap:**

Permette agli item di andare a capo se non c'è spazio.

- `nowrap` (default): tutti gli item su una sola riga
- `wrap` : gli item vanno a capo
- `wrap-reverse` : vanno a capo in ordine inverso

Esempio:

```
.container {  
  flex-wrap: wrap;  
}
```

- **justify-content:**

Allinea gli item lungo l'asse principale.

- `flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`

Esempio:

```
.container {  
  justify-content: space-between;  
}
```

- **align-items:**

Allinea gli item lungo l'asse trasversale.

- `stretch` (default), `flex-start`, `flex-end`, `center`, `baseline`

Esempio:

```
.container {  
  align-items: center;  
}
```

- **align-content:**

Allinea le righe del contenitore quando c'è spazio extra (solo se c'è wrapping).

- `stretch`, `flex-start`, `flex-end`, `center`, `space-between`, `space-around`

Esempio:

```
.container {  
  align-content: flex-end;  
}
```

Flexbox: proprietà item

order, flex-grow, flex-shrink, flex-basis, align-self.

Dettaglio delle proprietà degli item Flexbox

- **order:**

Imposta l'ordine degli item nel container. Il valore di default è 0.

Esempio:

```
.item {  
  order: 2;  
}
```

Gli item con valore più basso vengono visualizzati prima.

- **flex-grow:**

Determina quanto un item può crescere rispetto agli altri per occupare lo spazio disponibile.

Esempio:

```
.item {  
  flex-grow: 1;  
}
```

Un valore maggiore fa crescere di più l'item.

- **flex-shrink:**

Stabilisce quanto un item può ridursi se lo spazio è insufficiente.

Esempio:

```
.item {  
  flex-shrink: 2;  
}
```

Un valore maggiore fa restringere di più l'item.

- **flex-basis:**

Imposta la dimensione iniziale dell'item prima che lo spazio venga distribuito.

Esempio:

```
.item {  
  flex-basis: 150px;  
}
```

Può essere in px, %, ecc.

- **align-self:**

Permette di allineare singolarmente un item lungo l'asse trasversale, sovrascrivendo `align-items` del container.

Esempio:

```
.item {  
  align-self: flex-end;  
}
```

Queste proprietà consentono di controllare la crescita, la riduzione, la dimensione e l'allineamento di ogni singolo item all'interno di un container Flexbox.

Esempio pratico: pagina HTML con layout Flexbox

Esempio pratico: pagina HTML con layout Flexbox (HTML)

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Esempio Flexbox</title>
    <link rel="stylesheet" href="flexbox-layout.css" />
  </head>
  <body>
    <header>Header</header>
    <div class="flex-container">
      <nav class="sidebar">
        <h2>Menu</h2>
        <ul>
          <li>Home</li>
          <li>Servizi</li>
          <li>Contatti</li>
        </ul>
      </nav>
      <section class="main-content">
        <h1>Benvenuto nella pagina Flexbox</h1>
        <p>
          Questo layout utilizza Flexbox per disporre sidebar, contenuto
          principale e aside su una riga.
        </p>
      </section>
      <aside class="aside">
        <h3>Info</h3>
        <p>Colonna laterale destra</p>
      </aside>
    </div>
    <footer>Footer</footer>
  </body>
</html>
```

Esempio pratico: pagina HTML con layout Flexbox (CSS)

File CSS (`flexbox-layout.css`):

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
.flex-container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: stretch;  
  min-height: 100vh;  
}  
.sidebar {  
  background: #007bff;  
  color: #fff;  
  padding: 30px 20px;  
  width: 220px;  
}  
.main-content {  
  background: #f9f9f9;  
  flex: 1;  
  padding: 40px;  
}  
.aside {  
  background: #eee;  
  padding: 30px 20px;  
  width: 180px;  
}  
header,  
footer {  
  background: #333;  
  color: #fff;  
  text-align: center;
```

CSS Grid

Introduzione al layout a griglia.

CSS Grid Layout è un potente sistema bidimensionale che permette di creare layout complessi e flessibili, organizzando gli elementi in righe e colonne.

Vantaggi di CSS Grid

- Gestione semplice di layout complessi.
- Posizionamento preciso di elementi.
- Supporto nativo per righe e colonne.
- Responsività avanzata.

Grid: definizione delle colonne e righe

Proprietà principali

- `display: grid;` – attiva il layout Grid sul contenitore.
- `grid-template-columns` – definisce il numero e la larghezza delle colonne.
- `grid-template-rows` – definisce il numero e l'altezza delle righe.

Esempio:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 200px 1fr 2fr;  
  grid-template-rows: 100px 300px;  
  gap: 20px;  
}
```

```
<div class="grid-container">  
  <div>Box 1</div>  
  <div>Box 2</div>  
  <div>Box 3</div>  
</div>
```

Grid: posizionamento degli elementi

Proprietà di posizionamento

- `grid-column` – posizione e estensione sulle colonne.
- `grid-row` – posizione e estensione sulle righe.
- `grid-area` – nome dell'area assegnata.

Esempio:

```
.item1 {  
  grid-column: 1 / 3; /* dalla colonna 1 alla 3 */  
  grid-row: 1 / 2; /* dalla riga 1 alla 2 */  
}  
.item2 {  
  grid-column: 3 / 4;  
  grid-row: 1 / 3;  
}
```

```
<div class="grid-container">  
  <div class="item1">Header</div>  
  <div class="item2">Content</div>  
</div>
```

Grid: aree nominate

Utilizzo di `grid-template-areas`

Permette di definire aree leggibili e posizionare gli elementi tramite nome.

```
.grid-layout {  
  display: grid;  
  grid-template-columns: 150px 1fr 150px;  
  grid-template-rows: 80px 1fr 60px;  
  grid-template-areas:  
    "header header header"  
    "sidebar main aside"  
    "footer footer footer";  
  gap: 10px;  
}
```

```
.header {  
  grid-area: header;  
}  
.sidebar {  
  grid-area: sidebar;  
}  
.main {  
  grid-area: main;  
}  
.aside {  
  grid-area: aside;  
}  
.footer {  
  grid-area: footer;  
}
```

```
<div class="grid-layout">
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="main">Main Content</div>
  <div class="aside">Aside</div>
  <div class="footer">Footer</div>
</div>
```

Anteprima struttura:

Header	Header	Header
Sidebar	Main Content	Aside
Footer	Footer	Footer

Questo esempio mostra come disporre header, sidebar, contenuto principale, aside e footer usando CSS Grid.

Esempio pratico: layout con CSS Grid

File CSS (`grid-layout.css`):

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
.grid-layout {  
  display: grid;  
  grid-template-columns: 180px 1fr 180px;  
  grid-template-rows: 70px 1fr 50px;  
  grid-template-areas:  
    "header header header"  
    "sidebar main aside"  
    "footer footer footer";  
  height: 100vh;  
  gap: 15px;  
}  
.header {  
  background: #007bff;  
  color: #fff;  
  grid-area: header;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 1.5em;
```

```
.sidebar {  
  background: #f0f0f0;  
  grid-area: sidebar;  
  padding: 20px;  
}  
.main {  
  background: #fff;  
  grid-area: main;  
  padding: 30px;  
}  
.aside {  
  background: #eee;  
  grid-area: aside;  
  padding: 20px;  
}  
.footer {  
  background: #333;  
  color: #fff;  
  grid-area: footer;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

File HTML:

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Esempio CSS Grid</title>
    <link rel="stylesheet" href="grid-layout.css" />
  </head>
  <body>
    <div class="grid-layout">
      <div class="header">Header</div>
      <div class="sidebar">Sidebar</div>
      <div class="main">Contenuto principale</div>
      <div class="aside">Aside</div>
      <div class="footer">Footer</div>
    </div>
  </body>
</html>
```

Grid: proprietà aggiuntive

- `gap` – spazio tra righe e colonne.
- `justify-items` – allineamento orizzontale degli item.
- `align-items` – allineamento verticale degli item.
- `justify-content` – allineamento della griglia nel contenitore.
- `align-content` – allineamento delle righe nel contenitore.

Esempio:

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
  justify-items: center;  
  align-items: start;  
}
```

Esercitazione: layout di card con CSS Grid

Crea una griglia di card responsive:

```
<div class="card-grid">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
  <div class="card">Card 4</div>
</div>
```

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 25px;
  padding: 30px;
}

.card {
  background: #fff;
  border: 1px solid #ddd;
  padding: 20px;
  box-shadow: 2px 2px 8px #eee;
  border-radius: 8px;
  text-align: center;
```

Grid: definizione delle colonne e righe

grid-template-columns, grid-template-rows.

Dettagli sulle unità di misura Grid

- Le colonne e le righe possono essere definite con unità fisse (`px` , `em` , ecc.), percentuali (`%`), frazioni (`fr`) o auto.
 - `1fr` indica una frazione dello spazio disponibile.
 - `auto` adatta la dimensione al contenuto.
 - Puoi combinare unità diverse:

```
grid-template-columns: 150px 2fr auto;
```

- La funzione `repeat()` semplifica la definizione di colonne/righe ripetute:

```
grid-template-columns: repeat(3, 1fr);
```

Crea 3 colonne uguali.

- Per layout responsive, usa `minmax()` :

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Le colonne si adattano al contenitore e al numero di elementi.

Risorse:

- [MDN: grid-template-columns](#)
- [MDN: grid-template-rows](#)

Esempio:

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-rows: 100px 200px;  
  gap: 20px;  
}
```

```
<div class="grid-container">  
  <div>Colonna 1, Riga 1</div>  
  <div>Colonna 2, Riga 1</div>  
  <div>Colonna 3, Riga 1</div>  
  <div>Colonna 1, Riga 2</div>  
  <div>Colonna 2, Riga 2</div>  
  <div>Colonna 3, Riga 2</div>  
</div>
```

Grid: posizionamento degli elementi

grid-column, grid-row, grid-area.

Esempi pratici di posizionamento Grid

- Posizionare un elemento su più colonne:

```
.item1 {  
  grid-column: 1 / 3; /* Occupa dalla colonna 1 alla 2 inclusa */  
}
```

- Posizionare un elemento su più righe:

```
.item2 {  
  grid-row: 2 / 4; /* Occupa dalla riga 2 alla 3 inclusa */  
}
```

- Utilizzare grid-area per assegnare un'area nominata:

```
.main-content {  
  grid-area: main;  
}
```

- Esempio HTML:

```
<div class="grid-container">  
  <div class="item1">Header</div>  
  <div class="item2">Sidebar</div>  
  <div class="main-content">Contenuto principale</div>  
  <div class="item4">Footer</div>  
</div>
```

- **Esempio CSS completo:**

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-rows: 80px 1fr 60px;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
  gap: 10px;  
}  
.item1 {  
  grid-area: header;  
}  
.item2 {  
  grid-area: sidebar;  
}  
.main-content {  
  grid-area: main;  
}  
.item4 {  
  grid-area: footer;  
}
```

Tipografia, font-size, font-weight, line-height, letter-spacing in CSS

font-family.

Esempio: proprietà tipografiche in CSS

```
.tipografia-demo {  
  font-family: "Roboto", Arial, sans-serif;  
  font-size: 2em;  
  font-weight: 700;  
  line-height: 1.5;  
  letter-spacing: 1px;  
}
```

```
<p class="tipografia-demo">  
  Questo testo usa font Roboto, dimensione 2em, grassetto, interlinea 1.5 e  
  spaziatura lettere di 1px.  
</p>
```

Font personalizzati con @font-face

```
@font-face {  
  font-family: "OpenSans";  
  src: url("OpenSans-Regular.ttf") format("truetype");  
}  
.custom-font {  
  font-family: "OpenSans", Arial, sans-serif;  
}
```

```
<p class="custom-font">Testo con font personalizzato OpenSans.</p>
```

Stili di testo

text-align, text-decoration, text-transform, text-shadow.

Esempio: stili di testo

```
.testo-demo {  
  text-align: center;  
  text-decoration: underline;  
  text-transform: uppercase;  
  text-shadow: 1px 1px 3px #888;  
  font-size: 1.5em;  
  font-weight: bold;  
  letter-spacing: 2px;  
}
```

```
<p class="testo-demo">  
  Questo testo è centrato, maiuscolo, sottolineato e con ombra.  
</p>
```

Immagini e background

background-color, background-image, background-size, background-position.

Esempio: background immagine e dimensionamento

```
.background-demo {  
  background-color: #f0f0f0;  
  background-image: url("https://images.unsplash.com/photo-1506744038136-46273834b3fb");  
  background-size: cover;  
  background-position: center;  
  width: 400px;  
  height: 200px;  
  border-radius: 12px;  
  margin-bottom: 20px;  
}
```

```
<div class="background-demo"></div>
```

Bordo e raggio di bordo

border, border-radius, border-style, border-width, border-color.

Esempio: bordo e raggio di bordo

```
.bordo-demo {  
  border: 2px solid #007bff;  
  border-radius: 12px;  
  padding: 20px;  
  width: 250px;  
  margin-bottom: 20px;  
}
```

```
<div class="bordo-demo">Questo box ha un bordo blu e angoli arrotondati.</div>
```

Ombre - box-shadow e text-shadow.

Esempio: ombra su box e testo

```
.box-shadow-demo {  
  width: 200px;  
  height: 100px;  
  background: #fff;  
  border: 1px solid #ddd;  
  box-shadow: 4px 4px 12px #888;  
  margin-bottom: 20px;  
}  
  
.text-shadow-demo {  
  font-size: 2em;  
  color: #007bff;  
  text-shadow: 2px 2px 4px #aaa;  
}
```

```
<div class="box-shadow-demo">Box con ombra</div>  
<p class="text-shadow-demo">Testo con ombra</p>
```

Transizioni CSS

Proprietà principali delle transizioni CSS

Le transizioni permettono di animare il cambiamento di una o più proprietà CSS in modo fluido.

- **transition-property:** indica quali proprietà saranno animate (es. `background-color`, `width`, `all`).
- **transition-duration:** durata della transizione (es. `0.5s`, `200ms`).
- **transition-timing-function:** curva di accelerazione (`ease`, `linear`, `ease-in`, `ease-out`, `cubic-bezier`).
- **transition-delay:** ritardo prima dell'inizio della transizione (es. `0.2s`).

Esempio base:

```
.button {  
  background: #007bff;  
  color: #fff;  
  transition-property: background, color;  
  transition-duration: 0.3s;  
  transition-timing-function: ease;  
}  
.button:hover {  
  background: #e83e8c;  
  color: #fff200;  
}
```

Esempio pratico: animare la larghezza di un box

```
.box {  
  width: 100px;  
  height: 100px;  
  background: #007bff;  
  transition: width 0.5s ease;  
}  
.box:hover {  
  width: 200px;  
}
```

Esercizio: transizione su un link

Crea un link che cambi colore e sottolineatura in modo animato al passaggio del mouse:

```
a {  
  color: #333;  
  text-decoration: none;  
  transition:  
    color 0.4s,  
    text-decoration 0.4s;  
}  
a:hover {  
  color: #007bff;  
  text-decoration: underline;  
}
```

Risorse:

- [MDN: Transizioni CSS](#)

Animazioni CSS

@keyframes, animation-name, animation-duration, animation-iteration-count.

Proprietà principali delle animazioni CSS

Le animazioni CSS permettono di creare effetti dinamici e complessi tramite la regola `@keyframes` e le proprietà di animazione.

- **@keyframes:** definisce gli stati intermedi dell'animazione.
- **animation-name:** nome dell'animazione da applicare.
- **animation-duration:** durata dell'animazione (es. `2s` , `500ms`).
- **animation-iteration-count:** quante volte l'animazione viene ripetuta (`1` , `infinite` , ecc.).
- **animation-timing-function:** curva di accelerazione (`ease` , `linear` , ecc.).
- **animation-delay:** ritardo prima dell'inizio.

Esempio base di animazione

```
@keyframes fade-in {  
  from {  
    opacity: 0;  
  }  
  to {  
    opacity: 1;  
  }  
}  
  
.box {  
  animation-name: fade-in;  
  animation-duration: 1s;  
  animation-iteration-count: 1;  
}
```

Esempio pratico: animazione di movimento

```
@keyframes slide-right {  
  0% {  
    transform: translateX(0);  
  }  
  100% {  
    transform: translateX(200px);  
  }  
}  
  
.movable {  
  width: 100px;  
  height: 100px;  
  background: #e83e8c;  
  animation: slide-right 1.5s ease-in-out infinite alternate;  
}
```

```
<div class="movable"></div>
```

Esercizio: animazione su un bottone

Crea un bottone che cambia colore e si ingrandisce al passaggio del mouse:

```
@keyframes grow {  
  to {  
    background: #007bff;  
    transform: scale(1.2);  
  }  
}  
  
.button-animato {  
  background: #e83e8c;  
  color: #fff;  
  padding: 15px 30px;  
  border: none;  
  transition: transform 0.3s;  
}  
  
.button-animato:hover {  
  animation: grow 0.4s forwards;  
}
```

Risorse:

- [MDN: Animazioni CSS](#)

Media Queries

Le **media queries** permettono di applicare regole CSS diverse in base alle caratteristiche del dispositivo (larghezza, altezza, orientamento, risoluzione, ecc.).

Sintassi base

```
@media (max-width: 600px) {  
  body {  
    background: #f0f0f0;  
  }  
}
```

Questa regola si applica solo se la larghezza della finestra è inferiore a 600px.

Principali tipi di media

- **screen**: dispositivi con schermo (desktop, tablet, smartphone)
- **print**: per la stampa su carta

Esempi pratici

```
/* Layout mobile */
@media screen and (max-width: 480px) {
  .menu-desktop {
    display: none;
  }
  .menu-mobile {
    display: block;
  }
}

/* Layout tablet */
@media screen and (min-width: 481px) and (max-width: 1024px) {
  .sidebar {
    width: 120px;
  }
}

/* Stampa */
@media print {
  body {
    background: #fff;
    color: #000;
  }
}
```

Proprietà comuni usate nelle media queries

- `max-width` / `min-width` : larghezza massima/minima della viewport
- `max-height` / `min-height` : altezza massima/minima
- `orientation` : `portrait` o `landscape`
- `resolution` : densità dei pixel (`dpi` , `dppx`)

Risorse

- [MDN: Media Queries](#)
- [CSS Tricks: Media Queries for Standard Devices](#)

Esempio generale: Media Query Mobile First

Il paradigma **mobile first** consiste nello scrivere prima gli stili per dispositivi mobili e poi aggiungere regole per schermi più grandi tramite media query.

Esempio:

```
/* Stili base: mobile */
.container {
  padding: 15px;
  font-size: 1em;
}

/* Tablet e desktop */
@media (min-width: 600px) {
  .container {
    padding: 30px;
    font-size: 1.2em;
  }
}

@media (min-width: 1024px) {
  .container {
    max-width: 900px;
    margin: 0 auto;
    font-size: 1.3em;
  }
}
```

HTML di esempio:

```
<div class="container">
  <h2>Layout Mobile First</h2>
  <p>Questa sezione si adatta a smartphone, tablet e desktop.</p>
</div>
```

Suggerimento:

Scrivi sempre prima gli stili per mobile, poi aggiungi media query con `min-width` per dispositivi più larghi.

Variabili CSS

Le **variabili CSS** (custom properties) permettono di definire valori riutilizzabili per colori, spaziature, font, ecc. Si dichiarano con la sintassi `--nome-variabile` e si usano con `var(--nome-variabile)`.

Esempio di dichiarazione e utilizzo:

```
:root {  
  --main-color: #007bff;  
  --padding-base: 20px;  
}  
  
.button {  
  background-color: var(--main-color);  
  padding: var(--padding-base);  
  color: #fff;  
}
```

In questo esempio, `--main-color` e `--padding-base` sono variabili definite a livello globale (`:root`).

Sovrascrittura delle variabili

Le variabili CSS possono essere sovrascritte in un sotto-albero della pagina, ad esempio su un elemento specifico:

```
.card {  
  --main-color: #e83e8c;  
  background-color: var(--main-color);  
  padding: var(--padding-base);  
}
```

Qui, `.card` usa una versione diversa di `--main-color`, ma mantiene il padding globale.

Esempio pratico: tema chiaro/scuro con variabili

```
:root {
  --bg-color: #fff;
  --text-color: #222;
}
body {
  background: var(--bg-color);
  color: var(--text-color);
}
body.dark-mode {
  --bg-color: #222;
  --text-color: #fff;
}
```

Attivando la classe `dark-mode` su `<body>`, i colori vengono sovrascritti.

Risorse:

- [MDN: Custom properties \(CSS variables\)](#)

Esercizio

Creiamo una pagina html con i due layout chiaro e scuro utilizzando le variabili CSS sopra descritte.

```
<button id="toggle-theme">Cambia tema</button>
<script>
  document.getElementById("toggle-theme").onclick = function () {
    document.body.classList.toggle("dark-mode");
  };
</script>
```

Specificità e cascata

Come il browser decide quali regole applicare.

Specificità: come funziona

La **specificità** determina quale regola CSS viene applicata quando più regole corrispondono allo stesso elemento. Ogni selettore ha un "peso" calcolato in base a:

- Selettori ID (`#id`): valore alto (100)
- Selettori classe, attributo e pseudo-classe (`.classe` , `[attr]` , `:hover`): valore medio (10)
- Selettori di tipo e pseudo-elemento (`div` , `::after`): valore basso (1)
- Selettore universale (`*`): valore 0

Esempio di calcolo:

```
#menu .item a {  
  color: red;  
}
```

La cascata

La **cascata** è il processo con cui il browser decide quale stile applicare:

1. Stili inline (nell'HTML) hanno priorità massima.
2. Poi vengono i fogli di stile esterni e interni.
3. In caso di pari specificità, vince la regola scritta più in basso (più recente).

Importante: !important

La dichiarazione `!important` sovrascrive la cascata e la specificità, ma va usata con cautela perché rende il codice difficile da mantenere.

Esempio:

```
p {  
  color: blue !important;  
}
```

Risorse

Importazione di CSS

@import e collegamento di file esterni.

Importazione di CSS

Collegamento di file CSS esterni

Per applicare uno stile CSS a una pagina HTML, si collega un file esterno tramite il tag `<link>` nell' `<head>` :

```
<link rel="stylesheet" href="stili.css" />
```

Importazione con `@import`

All'interno di un file CSS, puoi importare altri fogli di stile usando la regola `@import` :

```
@import url("reset.css");  
@import url("https://fonts.googleapis.com/css?family=Roboto");
```

Nota:

- Il metodo `<link>` è preferito per motivi di performance.
- `@import` può causare un caricamento più lento degli stili.

Ordine di importazione

Gli stili importati con `@import` vengono caricati prima delle regole scritte nel file.

Gli stili collegati con `<link>` vengono applicati nell'ordine in cui appaiono nell'HTML.

Risorse:

- [MDN: Collegare fogli di stile CSS](#)

Strumenti di sviluppo

Utilizzo degli strumenti del browser per il debug CSS.

Strumenti di sviluppo nei browser

I browser moderni (Chrome, Firefox, Edge, Safari) includono strumenti di sviluppo che permettono di:

- Ispezionare il DOM e vedere gli stili applicati a ogni elemento.
- Modificare il CSS in tempo reale per testare cambiamenti.
- Visualizzare la cascata e la specificità delle regole.
- Analizzare il box model, margini, padding e dimensioni.
- Debuggare problemi di layout e responsive design.
- Simulare dispositivi mobili e media queries.

Come aprire gli strumenti

- **Chrome/Edge:** tasto destro → "Ispeziona" oppure **F12**
- **Firefox:** tasto destro → "Analizza elemento" oppure **F12**

Funzionalità utili

- **Inspector:** naviga tra gli elementi HTML e visualizza gli stili CSS.
- **Console:** mostra errori, avvisi e permette di eseguire codice JavaScript.
- **Network:** analizza il caricamento dei file CSS e altre risorse.
- **Device Toolbar:** simula schermi di smartphone/tablet per testare il responsive.

Suggerimento

Utilizza gli strumenti di sviluppo per:

- Individuare rapidamente errori di stile.
- Testare modifiche senza dover aggiornare il file CSS.
- Comprendere come la cascata e la specificità influenzano il risultato finale.

Risorse:

- [MDN: Strumenti di sviluppo per il web](#)
- [Google Chrome DevTools](#)

Best practice CSS

Organizzazione, riutilizzo e mantenibilità del codice CSS.

Best practice CSS

- **Organizza il codice in sezioni logiche:** Raggruppa le regole per componenti, layout, utility, ecc.
- **Usa nomi di classi chiari e descrittivi:** Preferisci nomi che riflettano la funzione o il contenuto (`.btn-primary`, `.card-header`).
- **Evita l'uso eccessivo di ID e !important:** Favorisci le classi per una maggiore riusabilità e mantenibilità.
- **Scrivi CSS modulare:** Separa gli stili in più file (base, layout, componenti, temi).
- **Commenta il codice:** Usa commenti per spiegare sezioni complesse o scelte particolari.
- **Utilizza variabili CSS:** Centralizza colori, spaziature e font per facilitare modifiche globali.
- **Adotta una convenzione di naming:** Ad esempio BEM (Block Element Modifier) per strutturare le classi.

- **Minimizza e ottimizza il CSS:** Rimuovi regole inutilizzate e usa strumenti di minificazione per migliorare le performance.
- **Testa su diversi browser e dispositivi:** Verifica la compatibilità e il comportamento responsive.
- **Documenta le regole e i componenti:** Mantieni una documentazione aggiornata per facilitare la collaborazione.

Risorse:

- [MDN: Organizzazione e best practice CSS](#)
- [CSS Guidelines](#)

Flexbox vs CSS Grid

Differenze principali

- **Flexbox** è un sistema di layout monodimensionale: organizza gli elementi in una riga o in una colonna.
- **CSS Grid** è un sistema di layout bidimensionale: permette di gestire righe e colonne contemporaneamente.

Caratteristica	Flexbox	CSS Grid
Dimensione	Monodimensionale	Bidimensionale
Direzione	Riga o colonna	Righe e colonne
Posizionamento	Sequenziale	Posizionamento preciso
Allineamento	Ottimo su asse singolo	Ottimo su più assi
Layout complessi	Più difficile	Più semplice e potente

Quando usare Flexbox

- Allineamento e distribuzione di elementi su **una sola riga o colonna** (es. navbar, pulsanti, card in fila).
- Layout semplici e componenti UI che richiedono flessibilità su un asse.
- Quando l'ordine degli elementi può cambiare facilmente.

Esempi:

Barre di navigazione, gruppi di bottoni, layout di card orizzontali/verticali.

Quando usare CSS Grid

- Layout **complessi** che richiedono il controllo su **righe e colonne**.
- Pagine con aree principali (header, sidebar, main, footer) disposte in una griglia.
- Quando serve posizionare elementi in punti specifici della griglia.

Esempi:

Layout di pagina, dashboard, gallerie di immagini, griglie di card responsive.

Consiglio pratico

- **Flexbox**: scegli per layout lineari e componenti.
- **Grid**: scegli per strutture di pagina e layout bidimensionali.
- Puoi **combinare** Grid e Flexbox: ad esempio, usare Grid per il layout principale e Flexbox per i componenti interni.

Risorse:

- [Flexbox vs Grid su MDN](#)
- [CSS Tricks: When to use Flexbox and Grid](#)

Framework CSS bootstrap

Introduzione a Bootstrap

Bootstrap è uno dei framework CSS più popolari per creare layout responsive e componenti UI moderni in modo rapido.

Include una vasta gamma di classi predefinite per griglie, bottoni, form, alert, navbar e molto altro.

Come includere Bootstrap

Collega Bootstrap tramite CDN nel `<head>` della tua pagina HTML:

```
<link  
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"  
  rel="stylesheet"  
>
```

Griglia Bootstrap

Bootstrap usa un sistema a 12 colonne per creare layout flessibili e responsive.

Esempio di layout a due colonne:

```
<div class="container">  
  <div class="row">  
    <div class="col-8">Colonna principale</div>  
    <div class="col-4">Colonna laterale</div>  
  </div>  
</div>
```

Componenti base Bootstrap

Bottoni

```
<button class="btn btn-primary">Bottone primario</button>  
<button class="btn btn-secondary">Bottone secondario</button>
```

Alert

```
<div class="alert alert-success" role="alert">  
  Operazione completata con successo!  
</div>
```

Navbar

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <a class="navbar-brand" href="#">Logo</a>
  <button
    class="navbar-toggler"
    type="button"
    data-bs-toggle="collapse"
    data-bs-target="#menu"
  >
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="menu">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Servizi</a></li>
      <li class="nav-item"><a class="nav-link" href="#">Contatti</a></li>
    </ul>
  </div>
</nav>
```

Esempio pratico: pagina con Bootstrap

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8" />
    <title>Esempio Bootstrap</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
    />
  </head>
  <body>
    <nav class="navbar navbar-dark bg-primary mb-4">
      <div class="container-fluid">
        <a class="navbar-brand" href="#">Corso Web</a>
      </div>
    </nav>
    <div class="container">
      <div class="row">
        <div class="col-md-8">
          <h1>Benvenuto!</h1>
          <p class="lead">
            Questa pagina usa Bootstrap per layout e componenti.
          </p>
          <button class="btn btn-success">Azione</button>
        </div>
        <div class="col-md-4">
          <div class="alert alert-info">Sidebar con Bootstrap</div>
        </div>
      </div>
    </div>
    <footer class="bg-dark text-white text-center py-3 mt-4">
      Footer Bootstrap
    </footer>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  </body>
</html>
```

Risorse Bootstrap

- [Documentazione ufficiale Bootstrap](#)
- [Componenti Bootstrap](#)
- [Griglia Bootstrap](#)